



Hour 4 — Forms, Routing, HTTP & Testing Intro

Duration: 60 minutes **Topics:** Forms 15 min | Routing 15 min | HTTP Requests 15 min | Jest & RTL 15 min

Learning Objectives

By the end of this session, students will be able to:

- **Build controlled React forms**
- **Handle multiple form fields**
- **Navigate between pages using React Router**
- **Fetch REST API data from Spring Boot**
- **Render API data dynamically**
- **Understand `useEffect()` for API calls**
- **Write basic React component tests**
- **Understand the roles of Jest and React Testing Library**

Application Flow

React UI | — Forms | — Routing | — HTTP Requests | — Testing | ▼ Spring Boot [REST API](#)

Controlled Forms in React

A **controlled form** keeps the form values inside React state.

```
const [form, setForm] = useState({ name: "", email: "" });
```

The input value is controlled by React:

```
<input name="name" value={form.name} onChange={handleChange} />
```

Flow

User types ↓ `onChange()` ↓ `setForm()` ↓ React State Updated ↓ Component Re-renders ↓ `Input` displays new value

Why controlled forms?

React always knows the
current form value

Easy validation

Easy submission

Easy to reset

Easy to send data to an API

Handling Multiple Form Fields

Instead of creating separate state variables:

```
const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [phone, setPhone] = useState("");
```

Use one object:

```
const [form, setForm] = useState({ name: "", email: "", phone: "" });
```

Generic change handler

```
function handleChange(e) {
  setForm({ ...form, [e.target.name]: e.target.value });
}
```

Important concept

`[e.target.name]` allows the same handler to update different fields.

```
name → form.name
email → form.email
phone → form.phone
```

Form Submission & Validation

Forms normally trigger a browser page reload.

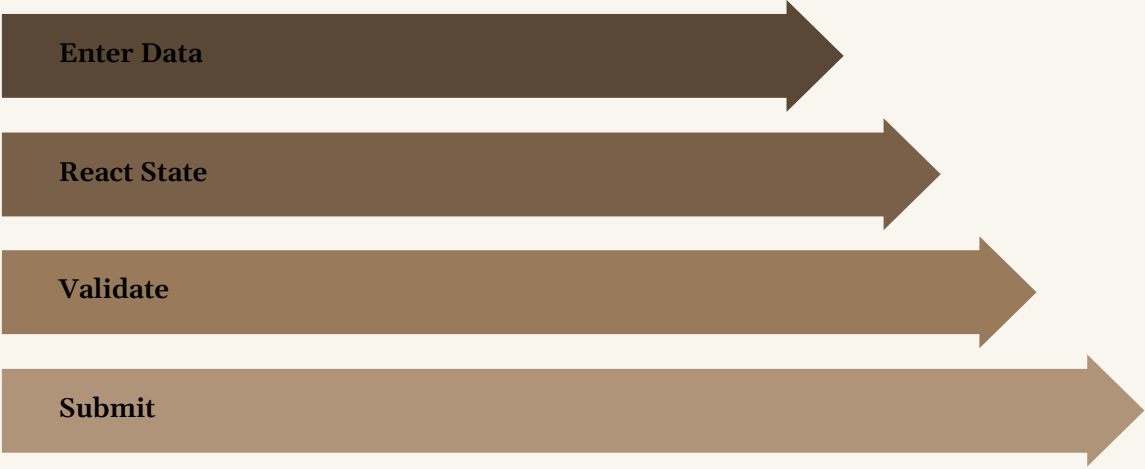
React prevents this using:

```
function handleSubmit(e) {  
  e.preventDefault();  
  console.log(form);  
}
```

Basic validation

```
if (!form.name) {  
  alert('Name is required');  
  return;  
}  
if (!form.email) {  
  alert('Email is required');  
  return;  
}
```

Complete flow



Enter Data

React State

Validate

Submit

React Router

React Router enables **client-side routing**.

Without routing

Browser ↓ Full [Page](#) Request ↓ Server

With React Router

URL Change ↓ React Router ↓ Matching Component ↓ UI Updated

Example:

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/students" element={<StudentList />} />
</Routes>
```

Navigation:

```
<Link to="/">Home</Link>
<Link to="/students">Students</Link>
```

Key point

The page does not need to reload when navigating between React routes.

Building a Multi-Page React Application

A simple Student Portal:

Student Portal | | Home | | Students | | | Student List | | | Student Details | | Register Student |
| About

Example routes:

```
<Route path="/" element={<Home />} />  
<Route path="/students" element={<StudentList />} />  
<Route path="/register" element={<RegisterForm />} />
```

Navigation

```
<nav>  
  <Link to="/">Home</Link>  
  <Link to="/students">Students</Link>  
  <Link to="/register">Register</Link>  
</nav>
```

HTTP Requests from React

React applications commonly communicate with backend APIs.

Example Spring Boot API:

```
GET http://localhost:8080/api/students
```

React:

```
useEffect(() => {  
  fetch('http://localhost:8080/api/students')  
    .then(res => res.json())  
    .then(data => setStudents(data))  
    .catch(err => console.error(err));  
}, []);
```

HTTP methods

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Delete data

React ↔ Spring Boot

useEffect() for API Calls

`useEffect()` runs side effects after rendering.

```
useEffect(() => {  
  fetchStudents();  
}, []);
```

The empty dependency array:

```
[]
```

means the effect runs once when the component mounts.

Typical pattern

```
const [students, setStudents] = useState([]);  
  
useEffect(() => {  
  fetchStudents();  
}, []);  
  
function fetchStudents() {  
  fetch('http://localhost:8080/api/students')  
    .then(response => response.json())  
    .then(data => setStudents(data));  
}
```

Why not fetch directly in the component body?

Because rendering should not repeatedly trigger API calls.

Displaying API Data

API response

```
[  
  { "id": 1, "name": "Rahul" },  
  { "id": 2, "name": "Priya" }  
]
```

React

```
<ul>  
  {students.map(student => (  
    <li key={student.id}>  
      {student.name}  
    </li>  
  ))}  
</ul>
```

Why key?

React uses `key` to identify individual list elements.

```
key={student.id}
```

Prefer a stable unique identifier.

Avoid:

```
key={Math.random()}
```

Introduction to Jest

Jest is a JavaScript testing framework.

Students previously learned:

Mocha + Chai

React testing:

Jest + React Testing Library

Example:

```
test('shows welcome message', () => {  
  render(<Welcome />);  
  expect(  
    screen.getByText('Student Portal')  
  ).toBeInTheDocument();  
});
```

Jest provides

- Test execution
- Assertions
- Test grouping
- Mocking capabilities
- Test reporting

Basic structure

```
test('description', () => {  
  // Arrange  
  // Act  
  // Assert  
});
```

React Testing Library

React Testing Library focuses on **user-visible behavior**.

```
render(<Welcome />);
```

Then query the UI:

```
screen.getByText('Student Portal');
```

Common queries:

```
getByText()  
getByRole()  
getByLabelText()
```

Example

```
const button = screen.getByRole(  
  'button',  
  { name: 'Register' }  
);
```

Testing philosophy

Don't ask:

"What internal React state does this component have?"

Ask:

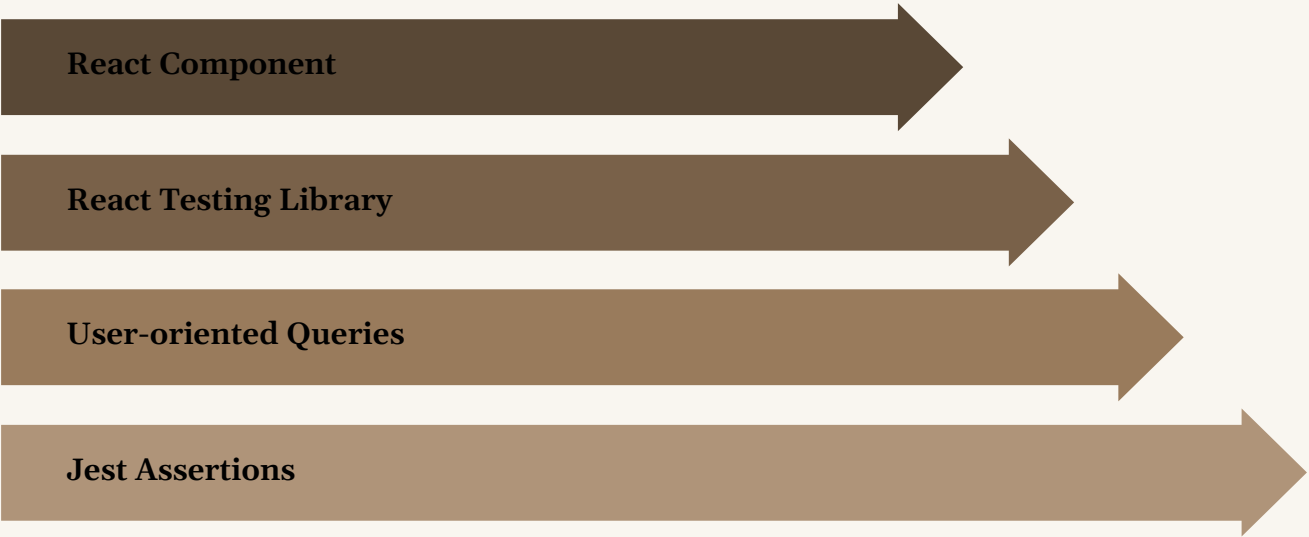
"What does the user see and interact with?"

Complete React Application & Testing Flow

Student Management Application

React Application |  ▼▼▼ Forms Routing Lists | | |
▼ HTTP Requests | ▼ Spring Boot [REST](#) API | ▼ Database

Testing



Key Takeaways

Forms → React state	Routing → React Router
API → <code>fetch()</code> / <code>Axios</code>	Side effects → <code>useEffect()</code>
Lists → <code>map()</code> + <code>key</code>	Tests → Jest + RTL